

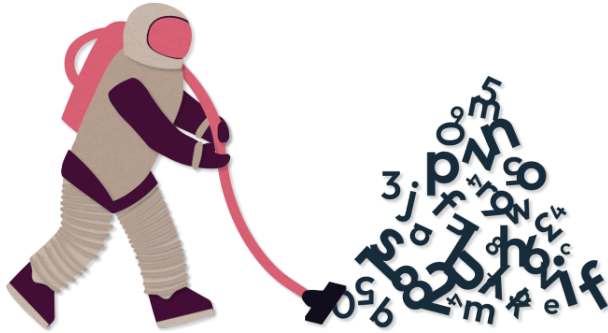
INTRODUCTION TO PYTHON



**Jumping
Rivers**

CREATING CLARITY WITH YOUR DATA.

Saving organisations like yours thousands of work hours.



Jumping Rivers is an analytics company whose passion is data and machine learning. We help our clients move from data storage to data insights. Our trainers and consultants come with over 100 years combined experience in R, Python, Stan, Scala and other programming languages.

Consultancy

- Posit (formerly RStudio) certified - the only UK partner.
- We have plans for managing all Posit products. From on-demand support to full care packages.
- As a Posit reseller, we can provide free consultancy.
- Experts in both data science and data engineering.
- Creation of bespoke dashboards and Shiny apps.
- Optimisation of your algorithms.



Training

- R/Python/Stan/Scala
- The Tidyverse
- Machine Learning and Tensorflow
- Dashboards with Shiny
- Statistical Modelling
- And many more!



hello@jumpingrivers.com

@jumping_uk

“FOLLOW. BUT! FOLLOW ONLY IF YE BE MEN OF VALOUR, FOR THE ENTRANCE TO THIS CAVE IS GUARDED BY A CREATURE SO FOUL, SO CRUEL, THAT NO MAN YET HAS FOUGHT WITH IT AND LIVED! BONES OF FULL FIFTY MEN LIE STREWN ABOUT ITS LAIR. SO, BRAVE KNIGHTS, IF YOU DO DOUBT YOUR COURAGE OR YOUR STRENGTH, COME NO FURTHER, FOR DEATH AWAITS YOU ALL WITH NASTY, BIG, POINTY TEETH.”

— *TIM THE ENCHANTER, MONTY PYTHON AND THE HOLY GRAIL.*

“THE TROUBLE WITH HAVING AN OPEN MIND, OF COURSE, IS THAT PEOPLE WILL INSIST ON COMING ALONG AND TRYING TO PUT THINGS IN IT.”

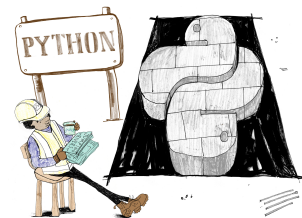
— *TERRY PRATCHETT, DIGGERS*

Introduction

0.1 Why use Python?

Python is a general purpose programming language that in recent years has gained *significant* popularity among data scientists and statisticians. A great swathe of Python packages have been developed which make it a breeze to handle, manipulate, and visualise data. Despite its zeal for data-wrangling, as Python is a *general purpose* language, it is easy to integrate with existing tools and languages which form part of your workflow.

Python also has a serious claim on the rights to be titled the world's most popular programming language. At the time of writing, the influential popularity rankings compiled by RedMonk and Tiobe both list Python in their top 2 language spot. As a result of this popularity, Python has an active and welcoming development community, with new packages being created and shared all the time.



0.2 Jupyter notebooks

Jupyter notebooks are an open-source web application that allow you to create and share documents that contain live code, equations, visualisations and narrative text. This makes them a good tool for exploring python when you are learning as well as a convenient way to share work with others.



0.3 Spyder

There are a number of different development environments for Python. Although Jupyter notebooks are useful for annotating code and for sharing results with colleagues, it is not an environment suited to more involved development, writing standalone scripts, programs or packages. For this we would want to use a more full-featured integrated development environment (IDE). Spyder is an IDE that provides an interactive Python terminal, as well as panes for managing projects, files and documentation. It comes shipped with the Anaconda distribution of Python.



Addendum

This course and all its associated materials were written using Python 3.12.3.

First Steps in Python



1.1 Simple operations

At its most basic, Python can be used as a calculator (step aside, Casio!). For example:

```
# Multiplication
5 * 5
#> 25

# Division
22 / 7
#> 3.142857142857143
```

In Python comments are denoted by the hash symbol (`#`). This means that everything proceeding a `#` is *ignored* by the Python interpreter. But if content prefixed by a `#` is ignored by Python, then why include comments at all?! We include comments to *explain* the intention of code to the reader, whether this reader is a colleague, sworn enemy or worse yet: your future self.

Along with multiplication (`*`) and division (`/`), Python also implements addition (`+`), subtraction (`-`), raise to the power of (`**`), modulo (`%`), and floor division (`//`).

1.2 Assignment operations

Typically, after loading data into Python or carrying out some computation, we want to store the resulting output to re-use later. We may do so by *assigning* this output to a *variable*. For example, to assign the value `5` to a variable named `x`:

```
x = 5
```

It is important to understand that we are assigning the thing on the *right* of the equals sign, to the thing on the *left* of the equals sign. Note that Python will not print anything to the screen when we run `x = 5`. To inspect the value of our variable we must type the variable name.

```
x
#> 5
```

We are free to choose variable names as we see fit. However, it is good practice to choose variable names which are descriptive, and indicative of their contents:



Figure 1.1: Comments, in Python preceeded by a `#`, can be used to document code for the reader. <https://xkcd.com/1421/>.

```
pi = 22 / 7
pi
#> 3.142857142857143
```

The *right hand side* of the assignment operator can contain anything which Python *already* knows about. In this way we can assign variables to other variables:

```
y = x
y
#> 5
```

Having assigned the variable `x` to variable `y`, what happens if we retrospectively alter `x`?

```
x = 10
y
#> 5
```

See that the value of `y` hasn't changed.¹

Although the assignment operator `=` is immediately reminiscent of the equals sign of mathematics, computer assignment and mathematical equality *should not be conflated*. Consider the following code chunk which sees us increment the value of `x`:

```
# Increment x to x + 1
x = x + 1
x
#> 11
```

Although perfectly valid Python, the statement `x = x + 1` is mathematically more troublesome...² In a similar fashion, we could increase by a factor of 2 as:

```
# Increase x by a factor of 2
x = 2 * x
x
#> 22
```

Python has a nifty shorthand for these sorts of operations:

```
# Equivalent to x = x + 1
x += 1
# Equivalent to x = x * 2
x *= 2
# Equivalent to x = x / 2
x /= 2
# Equivalent to x = x - 1
x -= 1
```

¹ This is because Python's integers are *immutable* objects: they can't be changed after creation. We will discuss immutability and mutability in more depth in Section 1.9.

² Hint: Try subtracting `x` from both sides of `x = x + 1` as if you were working with a *mathematical* equation.

1.3 Basic data types

There are a number of *built-in* data types of Python, ints (short for integers) being one of them. Python's ints represent the *whole* numbers: `1`, `2`, `3`, and up, and `0`, `-1`, `-2`, and down (I won't list them all here...). We can

use the `type()` function to find out the *type* of an object:

```
hours_since_coffee = 1
type(hours_since_coffee)
#> <class 'int'>
```

Python's float type can represent numbers which *aren't* whole numbers.³ You'll be able to tell a float from an integer by the presence of a decimal point.

```
root2 = 1.414
type(root2)
#> <class 'float'>
```

The string type (abbreviated to `str`) represents another built-in data type of Python. A string is defined by surrounding characters with quotation marks.⁴

```
best_cake = "carrot"
type(best_cake)
#> <class 'str'>
```

Python supports a binary logical data type called booleans (abbreviated to `bool`). These represent `True` and `False`.⁵

```
tropical_ncl = False
type(tropical_ncl)
#> <class 'bool'>
```

³ Typically, in programming languages floats are single precision numbers. In Python, however, a float is a *double* precision number.

⁴ Strings can be denoted using either single quotes `' '` or double quotes `" "`, so long as we are consistent with their usage.

⁵ Notice the capital letter: Python *is* case sensitive.

	Example		
Date type	1	2	3
int	2	42	144
float	2.71	42.0	3.141
str	"Euler"	'(1 + 5**0.5) / 2'	"cherry"
bool	True	False	

Table 1.1: Summary of the basic data types of Python.

1.4 Using functions

In programming, functions are used to incorporate a set of instructions that are to be used repeatedly. A function is a piece of code for carrying out a specific task. The task of the `type()` function, for example, is to return the type of data it is passed. You can identify functions by the:

- round brackets `()` following their name
- arguments given to the function: these are what the function needs to perform its task.

Some functions take multiple arguments. To help clarify what each value or variable represents in a function, arguments have names. You can view the help documentation of a function to find out what the arguments are named, by using the `help()` function

```
help(type)
```

Another useful function is the `print()` function. This function prints output to the screen:

```
print("and the clocks")
#> and the clocks
print("were striking")
#> were striking
print("thirteen.")
#> thirteen.
```

Chunk 1.1: Collect bonus points throughout this course by finding easter eggs and identifying cultural references. And remember: points make prizes.

1.5 Attributes and methods

Attributes and methods are *attached* to objects. We can think of it that

- Attributes are *variables* which are attached to an object
- Methods are *functions* which are attached to an object

Attributes are queried with the syntax `variable.attribute`, whereas methods are called with the syntax `variable.method()`. The important distinction between the two is the trailing round brackets `()` needed to call the method. Just like functions, methods may also take arguments. We will see examples of useful attributes later on.

An example of a method is the `.upper()` method associated with Python's string object:

```
quiet = "bang"
quiet.upper()
#> 'BANG'
```

Just like with functions, we can store the output returned by attributes and methods:

```
loud = quiet.upper()
loud
#> 'BANG'
```

The `dir()` function can be used to get a complete list of attributes and methods associated with an object. However, more convenient than `dir()` is *tab completion*. From a modern Python IDE simply type the name of the object you're interested in, followed by a fullstop, and then hit the <TAB> key to see a list of associated attributes and methods.

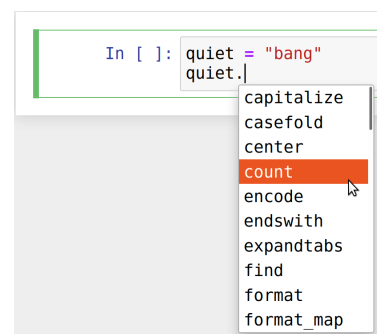


Figure 1.2: Tab-completion is a feature of most modern Python IDEs, and provides a convenient way to explore the attributes and methods of objects.

1.6 Lists

A list in Python represents a (mutable) collection of objects. Lists are constructed with square brackets `[]`. The entries of a list are separated by commas and specified inside the square brackets. We could create a list `x` containing the integers `1`, `2`, `3`, `4` and `5` as:

```
x = [1, 2, 3, 4, 5]
type(x)
#> <class 'list'>
```

A list can contain any mix of different data types:

```
y = [2, "be", "or", False, "2b"]
```

We can use the `len()` function (short for length) to find out the number of elements in our list:

```
len(y)
#> 5
```

Lists (and other objects, for that matter) can be sorted using the `sorted()` function:

```
y = [7, 6, 10, 8, 9]
z = sorted(y)
z
#> [6, 7, 8, 9, 10]
```

By default, `sorted()` sorts values in *ascending* order (smallest to largest). However, we can sort the values in *descending* order (largest to smallest) if we pass `reverse=True` to `sorted()`:

```
y = [7, 6, 10, 8, 9]
z = sorted(y, reverse=True)
z
#> [10, 9, 8, 7, 6]
```

1.7 List operations

We can use the `+` operator to concatenate lists together

```
x + y
#> [1, 2, 3, 4, 5, 7, 6, 10, 8, 9]
```

The `*` operator will repeat a list a specified number of times⁶

```
x * 2
#> [1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
```

⁶ The interactions between lists and the operators `+` and `*` may not feel particularly intuitive. You *may* have expected these operators to perform *elementwise* arithmetic on the elements of the lists. In Chapter 2 we will be introduced to a data type in which arithmetic operations *are* performed in an elementwise fashion.

1.8 Extracting elements from lists

We use square brackets, `[]`, to extract elements from lists; specifying the index of the element which we'd like to extract:

```
x[0]
#> 1
```

In Python `0` represents the *first* element of a list. So if we wanted the third element of `x`:

```
x[2]
#> 3
```

In general, to extract the `n`-th element of a list `x` we would use: `x[n-1]`.

Extraction: Negative indexing

We can also index lists with *negative* values. When we use negative indices we count *backwards* from the *end* of the list. We can use negative indexing

to extract the last element of a list as:

```
x[-1]
#> 5
```

Similarly we can extract the second to last element of the list `x` as `x[-2]` and the third to last element as `x[-3]` and so on. See Table 1.2 for a comparison between positive and negative indexing.

List	[1,	2,	3,	4,	5]
Positive Index	0	1	2	3	4
Negative Index	-5	-4	-3	-2	-1

Table 1.2: Position indexes for a list.

Extraction: Slicing

We can extract multiple values from lists with *slicing*. We do this using the `:` operator.⁷

We can use slicing to extract the values from index `start` through to the index `stop-1` as `x[start:stop]`. For instance, to extract `x[1]`, `x[2]` and `x[3]` from `x`:

```
# Extract x[1], x[2] and x[3]
x[1:4]
#> [2, 3, 4]
```

It can be helpful to think of the `start` and `stop` indices as referring to *boundaries* between the list elements (see Figure 1.3). Only elements between the two boundaries are extracted.

If we *don't* specify a value for `stop` (ie. `x[start:]`) then Python *assumes* we want to slice through to the *end* of the list:

```
# Extract x[2], x[3] and x[4] (x[4] is the last value in x)
x[2:]
#> [3, 4, 5]
```

Similarly, if we *don't* specify a value for `start` (ie. `x[:stop]`) then Python *assumes* we want to slice from the *beginning* of the list:

```
# Extract x[0], x[1], and x[2]
x[:3]
#> [1, 2, 3]
```

In this manner, if we don't specify a value for `start` or `stop`, we will get back our entire list:

```
# Returns the full list
x[:]
#> [1, 2, 3, 4, 5]
```

There is also an optional `step` value, which can be used in combination with any of the above patterns. With this, `x[start:stop:step]` extracts the values from `start`, not past `stop`, in increments of `step`:

```
# Extracts x[1] and x[3]
x[1::2]
#> [2, 4]
```

⁷ Note that when slicing lists the end point is not returned. The key point to remember is that the `:stop` value represents the first value that is *not* in the selected slice.

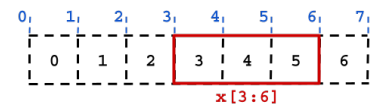


Figure 1.3: To extract cells 3 to 5, we slice between the left boundaries of cells 3 and 6.

Multi assignment

It is possible to assign values to multiple objects at once by extracting values from lists:

```
a, b, c = y[:3]
print(a, b, c)
#> 7 6 10
```

1.9 Mutability vs. immutability

Whenever we create an object Python must allocate it a place to live in memory. Variable names then *point* to this location in memory. In this way we can use variables to store values for re-use later. We can use the `id()` function to query the location of an object in memory:

```
x = 5
id(x)
#> 136028700758088
```

Python’s integers are *immutable*. This means that we *can’t change* the value of an integer *in place*. Therefore, if we wanted to increment the value stored in `x`, Python will have to assign a new piece of memory to store the incremented value in:

```
id(x)
#> 136028700758088
x = x + 1
id(x)
#> 136028700758120
```

The output of the `id()` function shows that `x` now points to a *different* location in memory. This is immutability. In Python integers, floats, strings and booleans are all immutable.

Python’s lists, however, are *mutable*. This means that we *can change* the values of a list *in place*: *without* the need for Python to assign it a new location in memory:

```
y = [1, 2, 3, 4, 5]
id(y)
#> 136029944613184
y[2] = 10
id(y)
#> 136029944613184
```

This time we have altered `y` *in place*: our list has changed without changing location in memory. If we assign two variables to the same object, the variables will both point to the same location in memory:

```
z = y
id(y)
#> 136029944613184
id(z)
#> 136029944613184
```

Type	Mutable	Immutable
int		✓
float		✓
string		✓
bool		✓
list	✓	
tuple		✓
dict	✓	

Table 1.3: The most commonly used Python types and their mutability / immutability status. Note that we will be introduced to dicts later.

Because lists are mutable, and because `y` and `z` point to the same location in memory, if we change the values in one variable, then the other variable will change too:

```
print(y, z)
#> [1, 2, 10, 4, 5] [1, 2, 10, 4, 5]
y[0] = 999
print(y, z)
#> [999, 2, 10, 4, 5] [999, 2, 10, 4, 5]
```

This behaviour may surprise you, and often catches many Python beginners out. To avoid this behaviour we use the `.copy()` method to ensure our two variables point to different locations in memory:

```
z = y.copy()
id(y)
#> 136029944613184
id(z)
#> 136029793600256
```

With this, if we change one variable, the other will remain unchanged:

```
y[0] = 1
print(y, z)
#> [1, 2, 10, 4, 5] [999, 2, 10, 4, 5]
```

NumPy

2.1 But first, what is a package?

A package is a collection of Python *modules*, where a *module* is a single file containing Python code. Packages, amongst other things, may define functions, variables, and classes. Most packages you will use are hosted on PyPI (pronounced “pie pea eye”). `pip` is the preferred installer program for Python packages, and can be used to install packages from PyPI. From the command line we install packages with: `pip install packagename`.

Importing packages

To import an installed package we use the `import` keyword:

```
import packagename
```

To access objects from the imported package we reference the imported namespace:

```
packagename.thing1
```

Aliasing imports

It is common to use imported packages repeatedly in our code. When we do so, typing out long package names over and over quickly becomes tedious. To alleviate this pain we can *alias* our imports. Aliasing allows us to pick some shorter name with which to refer to our imported package. We use the `as` keyword to alias our imports:

```
import packagename as pkg
```

We can now use our shorthand name to access objects within our package:

```
pkg.thing2
```

2.2 Introduction to NumPy

NumPy defines objects for storing large, multi-dimensional arrays and matrices, and provides high-level mathematical functions to operate on these objects. In doing so NumPy brings the computational performance of languages like C and Fortran into the comfort of Python. Whether they know it or not, almost every scientist working in Python utilises the power of NumPy.



Figure 2.1: PyPI (an abbreviation of Python Package Index) is a repository of Python packages.



2.3 NumPy arrays

NumPy's main object is the `ndarray`. NumPy's array object is *similar* to Python's list object, but is much better suited to data-oriented tasks.

It is convention to alias NumPy to `np` at import:

```
import numpy as np
```

We can construct NumPy arrays from lists:

```
y_list = [1, 2, 3, 4, 5]
y_array = np.array(y_list)
y_array
#> array([1, 2, 3, 4, 5])
```

We can use the `type()` function to see that `y_array` really is a data type we've not seen thus far:

```
type(y_array)
#> <class 'numpy.ndarray'>
```

In the same way as lists, arrays are also mutable:

```
y_array[0] = 5
y_array
#> array([5, 2, 3, 4, 5])
```

As with lists, we must be cautious of mutability when assigning two variables to the same array (Section 1.9). Just as with lists, arrays have a `.copy()` method which we can use to circumvent any potential mutability-whoopsies.

Homogeneity

All the entries of a NumPy array must be of *the same type*. In this way we say that NumPy arrays are *homogeneous*. Recall that lists can contain any mix of *different* data types (they are *heterogeneous*). The array's `.dtype` attribute tells us what type of data is stored within our array:

```
y_array.dtype
#> dtype('int64')
```

This tells us that `y_array` contains integers. The homogeneity restraint of NumPy's array may initially seem like a drawback, *however*, it is this restriction which allows NumPy to be so computationally performant.

Elementwise operations

With lists the `+` operator performed concatenation and the `*` operator performed repetition. However, when used on NumPy arrays these operators perform *elementwise* arithmetic:

```
x = np.array([1, 2, 3])
# Addition of arrays is performed elementwise
x + x
#> array([2, 4, 6])
```

Chunk 2.1: Although stylised as NumPy, in code we refer to the package in lower case as `numpy`.

Chunk 2.2: The `64` in `'int64'` represents that the integers are 64-bit double precision integers.

```
# Multiplication of arrays is performed elementwise
x * x
#> array([1, 4, 9])
```

```
# Division of arrays is performed elementwise
x / x
#> array([1., 1., 1.])
```

NumPy provides a myriad of mathematical functions, the majority of which act on arrays in an elementwise fashion:¹

```
# Exponentiate every entry in x
np.exp(x)
#> array([ 2.71828183,  7.3890561 , 20.08553692])
```

```
# Take the (natural) logarithm of every entry in x
np.log(x)
#> array([0.          , 0.69314718, 1.09861229])
```

```
# Take the sine of every entry in x
np.sin(x)
#> array([0.84147098, 0.90929743, 0.14112001])
```

¹ Notable exceptions to this rule are functions which perform aggregative calculations, such as `np.sum()`, `np.mean()`, `np.min()` and `np.max()`.

Broadcasting

Broadcasting refers to how NumPy performs operations on arrays and scalars², and arrays of different shapes. Subject to certain constraints, the smaller array (or scalar) is “broadcast” across the larger array so that their shapes are compatible to perform the desired operation:

```
# 100 is 'broadcast' across x
x * 100
#> array([100, 200, 300])
```

² This is just array / matrix speak for objects of type `int` and `float`.

2-d arrays, and beyond!

An array’s `.ndim` attribute tells us how many *dimensions* it has. So far we have only seen arrays with one dimension:

```
x
#> array([1, 2, 3])
x.ndim
#> 1
```

We can construct arrays with more than one dimension using lists of lists:

```
# Constructing a 4x2 array from a list of lists
x = np.array([[1, 2],
              [3, 4],
              [5, 6],
              [7, 8]])
x.ndim
#> 2
```

The `.shape` attribute of an array reports the number of entries in each of

its dimensions—for a 2-d array the `.shape` attribute represents the number of rows and columns:³

```
x.shape
#> (4, 2)
```

³ A common use of multi assignment (as introduced at the end of Section 1.8) is in extracting the number of rows and columns of an array from its `.shape` attribute:

```
nrow, ncol = x.shape .
```

Extracting elements from arrays

We can extract elements from arrays in much the same manner as we extracted elements from lists (see Section 1.8 for a recap); using square brackets `[]`; the slice operator `:`, and indexing from zero:

```
y = np.array([1, 2, 3, 4, 5])
# Extract the first element of y
y[0]
#> np.int64(1)
```

```
# Extract y[1], y[2] and y[3]
y[1:4]
#> array([2, 3, 4])
```

```
# Extract the first three entries of y
y[:3]
#> array([1, 2, 3])
```

We can extract values from arrays with two dimensions by specifying the row and column number of the entry we'd like. We specify the row number first, followed by the column number (`x[row, col]`):

```
x = np.array([[1, 2],
              [3, 4],
              [5, 6],
              [7, 8]])
# Extract the element of x in row 0, column 1
x[0, 1]
#> np.int64(2)
```

```
# Extract the element of x in row 2, column 0
x[2, 0]
#> np.int64(5)
```

We can use the slice operator `:` to extract entire rows and columns:

```
# Extract all the entries in column 1 of x
x[:, 1]
#> array([2, 4, 6, 8])
```

```
# Extract all the entries in row 2 of x
# Equivalent to x[2]
x[2, :]
#> array([5, 6])
```

Just as with lists, we can also use the slice operator to extract ranges of values:

```
# Extract entries in column 1 for rows 1, 2, and 3  
x[1:4, 1]  
#> array([4, 6, 8])
```

Dictionaries and Pandas

3.1 Dictionaries

Dictionaries (abbreviated to dict) are mutable objects used for storing *key-value pairs*. Dictionaries are constructed with curly braces: `{}`. Each *key* is mapped to a corresponding *value* with a colon `:`. Key-value pairs are separated by commas:

```
trainer = {
    "name": "Jonas",
    "age": 103,
    "likes indexing from zero": True
}
type(trainer)
#> <class 'dict'>
```

Here we have constructed a dictionary and assigned it to a variable named `trainer`. This dictionary has three key-value pairs. The keys are `"name"`, `"age"` and `"likes indexing from zero"` and the corresponding values are `"Jonas"`, `103`, and `True`. The values stored in our dictionary can be anything we like (ints, floats, bools, strings, or even other dictionaries!).

Extracting values

Like lists and arrays, we extract values from dictionaries with square brackets `[]`. Unlike lists and arrays we extract values from dictionaries with *keys* not *indices*. For example, if you wanted to know my age you could use the `"age"` key as:

```
trainer["age"]
#> 103
```

Similarly, you could check whether I'm a sane and respectable member of society with:

```
trainer["likes indexing from zero"]
#> True
```

Chunk 3.1: We could have wrote the key-value pairs on the same line, however, using multiple lines improves readability.

The keys don't *have* to be strings, although they often are, but they do have to be unique (and hashable).

3.2 Pandas

The Pandas library is a data-manipulation package built on top of NumPy. Its aim is to provide high performance, easy-to-use data structures and data



analysis tools. By convention we alias the Pandas package to `pd` at import:

```
import pandas as pd
```

Pandas has two key data structures: the `Series` and the `DataFrame`. As we will see, these objects behave as some amalgamation of a NumPy array and a dictionary.

3.3 Pandas Series

The Pandas `Series` object is a *labelled* one-dimensional array. In general, we may instantiate a `Series` as:

```
series = pd.Series(data, index=index)
```

Here `data` may be a list, dictionary, or one-dimensional NumPy array. The `index` argument passed to `pd.Series()` is optional, and represents labels which point to elements of the `Series`. If the `index` argument is not specified then Pandas will choose some sensible labels:

```
series = pd.Series([0.1, 0.2, 0.3])
series
#> 0    0.1
#> 1    0.2
#> 2    0.3
#> dtype: float64
```

Here we created a `Series` from a list. As we did not explicitly specify an index Pandas sensibly opted to label the values `0`, `1`, and `2`. We can label our `Series` with strings at creation as:

```
series = pd.Series([0.1, 0.2, 0.3], index=["a", "b", "c"])
series
#> a    0.1
#> b    0.2
#> c    0.3
#> dtype: float64
```

The `pd.Series()` function can be used to create `Series` objects from dictionaries:

```
exciting_dict = {"a": 0.1,
                 "b": 0.2,
                 "c": 0.3}
exciting_series = pd.Series(exciting_dict)
exciting_series
#> a    0.1
#> b    0.2
#> c    0.3
#> dtype: float64
```

When creating a `Series` from a dictionary, the index of the `Series` defaults to the keys of the dictionary. We will see how to extract values from `Series` objects in Section 3.1

3.4 Pandas DataFrames

If a `Series` is a hybrid of a dictionary and a one-dimensional NumPy array, then the `DataFrame` is a hybrid of a dictionary and a *two*-dimensional NumPy array.

The Pandas `DataFrame` object is reminiscent of the all-too-familiar spreadsheet; where rows typically represent observations, and columns typically represent variables. We can create a `DataFrame` :

- 1) By reading data in from a file—to be discussed in more detail in Chapter 5.
- 2) From a dictionary where each key represents a variable, and the corresponding values represent the associated observations.

For example, let's create a dictionary called `the_north` containing three key-value pairs:

```
the_north = {
    "city": ["Manchester", "York", "Leeds", "Birmingham"],
    "population": [553000, 210000, 789000, 1145000],
    "area": [115.6, 271.9, 551.7, 267.8]
}
```

We can then convert this dictionary to a `DataFrame` with the `pd.DataFrame()` function:

```
df = pd.DataFrame(the_north)
df
#>      city  population   area
#> 0  Manchester    553000  115.6
#> 1      York     210000  271.9
#> 2     Leeds     789000  551.7
#> 3 Birmingham   1145000  267.8
```

See that the *keys* of our dictionary (`"city"`, `"population"`, and `"area"`) have become *column names* in the resulting `DataFrame`.

The `.set_index()` method allows us to change the row labels (the index) in our `DataFrame`. We may pass a list or array to `.set_index()`, or we may pass the name of an existing column:

```
df = df.set_index("city")
df
#>      population   area
#> city
#> Manchester    553000  115.6
#> York          210000  271.9
#> Leeds          789000  551.7
#> Birmingham   1145000  267.8
```

Just as with NumPy arrays, we can use the `.shape` attribute to check the number of rows and columns in our `DataFrame` :

```
df.shape
#> (4, 2)
```

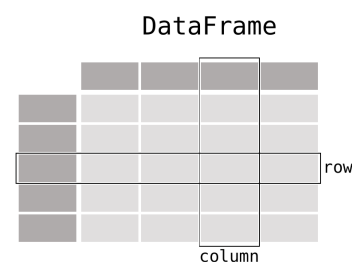


Figure 3.1: The Pandas `DataFrame` object has labelled columns and rows, much like a traditional spreadsheet.

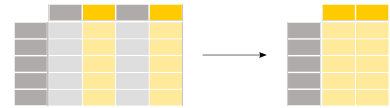
Chunk 3.2: In this dictionary we have three key-value pairs. The keys are strings, and their associated values are lists of length 4.

Chunk 3.3: The `.set_index()` method *does not* operate on `DataFrame`s in-place, and so we must assign its output back to `df`. Alternatively, we could pass the `inplace=True` argument to `.set_index()` to make this operation happen in-place.

Extracting data

The Python and NumPy index operators `[]`, can be used to extract values from `Series` and `DataFrame` objects. We can use this notation to extract columns from our `DataFrame`s as:

```
# Extract the area column
df["area"]
#> city
#> Manchester    115.6
#> York          271.9
#> Leeds         551.7
#> Birmingham   267.8
#> Name: area, dtype: float64
```



However, Pandas also provides the `.loc` and `.iloc` attributes for optimised data access, which we should use these for more involved data extraction:

- `.loc` is primarily used for *label*-based extraction.
- `.iloc` is primarily used for *index*-based extraction.

The `.loc` attribute can be used to extract values from our `DataFrame` with row and column *labels*. Recall from Section 2.3 that when indexing a two-dimensional object we specify the row we want first, and then the column:

```
# Extract the area (column label) of York (row label)
df.loc["York", "area"]
#> np.float64(271.9)
```

The slice operator `:` can be used to take slices of the `.loc` attribute. When slicing the `.loc` attribute it is important to note that both the start value *and* the stop value are returned—this behaviour is in contrast to the behaviour when slicing lists and NumPy arrays, in which the stop-value is not returned.

```
# Extract the population of the rows between
# and including Manchester and Leeds
df.loc["Manchester":"Leeds", "population"]
#> city
#> Manchester    553000
#> York          210000
#> Leeds         789000
#> Name: population, dtype: int64
```

The `.iloc` attribute can be used to extract values from our `DataFrame` with row and column *indices*. Indexing `iloc` behaves the same as indexing Python's lists and NumPy's arrays:

```
# Extract the entry in row 2 column 1
df.iloc[2, 1]
#> np.float64(551.7)
```

When we take slices of the `.iloc` attribute the end-point is *not* returned.

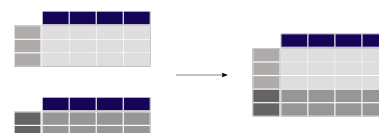
This conforms with the behaviour of Python's list type and NumPy's array type, and contrasts with the behaviour of `.loc` :

```
# Extract df[0, 0] and df[1, 0]
df.iloc[:2, 0]
#> city
#> Manchester      553000
#> York            210000
#> Name: population, dtype: int64
```

Appending rows

There are a number of ways to add new rows to an existing `DataFrame`. Notable solutions include using the `.append()` method and the `pd.concat()` function. However, the simplest solution is to specify the name of our new row with the `.loc` attribute, and assign our values to it:

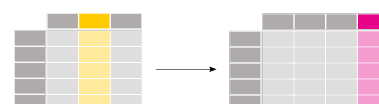
```
# Create a new row "Belfast" from a dictionary
df.loc["Belfast"] = {"population": 280000, "area": 115}
df
#>           population  area
#> city
#> Manchester      553000  115.6
#> York            210000  271.9
#> Leeds           789000  551.7
#> Birmingham     1145000  267.8
#> Belfast         280000  115.0
```



Appending columns

To append a new column to an existing `DataFrame` we specify the name of our new column, and the values which should go in it:

```
# Create a new column 'density'
df["density"] = df["population"] / df["area"]
df
#>           population  area  density
#> city
#> Manchester      553000  115.6  4783.737024
#> York            210000  271.9   772.342773
#> Leeds           789000  551.7  1430.125068
#> Birmingham     1145000  267.8  4275.578790
#> Belfast         280000  115.0  2434.782609
```



Removing rows and columns

Great, so we can create `DataFrame`s, extract values and create new rows and columns. But, what about removing rows and columns? I mean, let's be honest, Birmingham isn't *really* "the north" now is it? We can remove this row with the `.drop()` method:

```
df = df.drop("Birmingham")
df
#>           population    area    density
#> city
#> Manchester    553000  115.6  4783.737024
#> York          210000  271.9   772.342773
#> Leeds         789000  551.7  1430.125068
#> Belfast       280000  115.0  2434.782609
```

By default, the `.drop()` method assumes that we wish to remove rows. So, if we wish to remove columns, we have to explicitly tell the `.drop()` method this with the optional `axis` argument:

```
df = df.drop("population", axis="columns")
df
#>           area    density
#> city
#> Manchester  115.6  4783.737024
#> York        271.9   772.342773
#> Leeds       551.7  1430.125068
#> Belfast     115.0  2434.782609
```

Summaries

Many of the functions from the NumPy package can be applied to Pandas `DataFrame`s as methods. These operate on the `DataFrame` in a column-wise manner.

For example, the column means can be computed with:

```
df.mean()
#> area          263.550000
#> density       2355.246868
#> dtype: float64
```

Additionally, these methods can also be applied to specific columns of our `DataFrame`s:

```
df["area"].min()
#> np.float64(115.0)
```

There are a great number of NumPy functions and corresponding `DataFrame` methods which we have access to. A summary of these methods is presented in Table 3.1.

Method	Description
<code>.head()</code>	Preview the first 5 rows
<code>.sum()</code>	Sum the values in each column
<code>.min()</code>	Find the minimum value in each column
<code>.max()</code>	Find the maximum value each column
<code>.mean()</code>	Compute the mean of each column
<code>.std()</code>	Compute the standard deviation in each column
<code>.median()</code>	Compute the median of each column

Table 3.1: An overview of a few of the methods which can be used to summarise the columns of a Pandas `DataFrame`.

It's worth noting that NumPy functions can also be applied to Pandas DataFrames:

```
import numpy as np

np.mean(df, axis=0)
#> area      263.550000
#> density    2355.246868
#> dtype: float64
```

The `axis=0` argument means we care about *column* means. To extract *row* means we would use `axis=1`.

Graphics

Many different visualisation packages have been developed for Python. Probably the longest established of these packages is Matplotlib. Along with its standing as a pillar of visualisation in the Python world comes comprehensive documentation, an active development community, and a reliable and stable plotting interface.

4.1 Matplotlib

Matplotlib is a sprawling and featureful package. In fact, Matplotlib has a number of different interfaces which we may use to construct plots. By far the most popular of these interfaces—and certainly the easiest to use—is the `pyplot` interface. By convention we import this interface as `plt`:

```
import matplotlib.pyplot as plt
```

Basic plots

`plt.plot()` is a versatile function, and can be used to create line and scatter plots. For example, to plot the temperature against the number of ice creams consumed in the Jumping Rivers' offices:¹

```
temp = [12, 18, 21, 25, 30]
ice_creams = [1, 3, 9, 14, 5]

plt.plot(temp, ice_creams)
```

Axis labels

So, we have created our line plot (Figure 4.1), but it still leaves much to be desired. For example, faced *only* with the plot, how would a reader know what we're plotting? *Temperature vs. ant infestations?* Slander, I say; we have *hardly any* ant infestations. Let's make it clear that we're interested in temperate and ice creams by labelling our axes:

```
plt.plot(temp, ice_creams)

plt.xlabel("Temperature (degrees)")
plt.ylabel("Ice creams consumed")
```

matplotlib

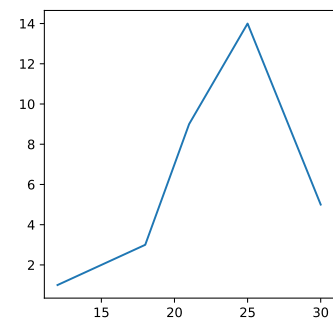


Figure 4.1: Using `plt.plot()` to create a simple line plot.

¹ No employees were harmed in the performance of this experiment.

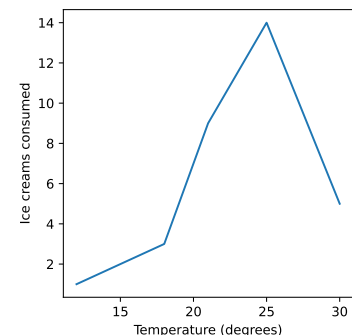


Figure 4.2: We can use `plt.xlabel()` and `plt.ylabel()` to add axis labels to our plots.

Axis limits

Along with labelling our axes, we may also set the limits of the x and y axes with the `plt.xlim()` , and `plt.ylim()` functions respectively:

```
plt.plot(temp, ice_creams)
plt.xlabel("Temperature (degrees)")
plt.ylabel("Ice creams consumed")

plt.xlim(10, 35)
#> (10.0, 35.0)
plt.ylim(0, 14)
#> (0.0, 14.0)
```

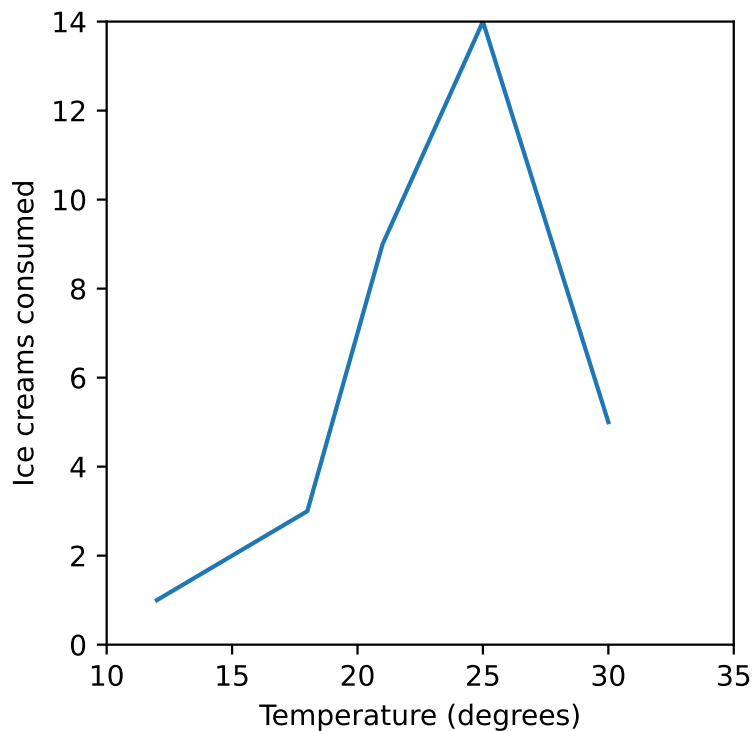


Figure 4.3: We can use `plt.xlim()` and `plt.ylim()` to change the limits of our axes.

Markers, lines, and colours

We may change how our data is displayed by passing a third *optional* argument to `plt.plot()` . This optional parameter provides a convenient way to alter the colours, markers, and lines used in our plot. This argument is specified as a *string* of the form:

```
[marker] [line] [colour]
```

For example, to plot our data with *crosses* coloured *red*, and a *dotted* line, we may use:

```
plt.plot(temp, ice_creams, 'x:r')

plt.xlabel("Temperature (degrees)")
```

```
plt.ylabel("Ice creams consumed")
```

In the above example our format string specified that we should use cross markers (`'x'`), coloured red (`'r'`), and a dotted line (`'.'`); hence `'x:r'` . The available markers, lines and colours are summarised in Tables 4.1 to 4.3.

Style sheets

We can change multiple aspects of our plot's aesthetics with *style sheets*. You can see a list of our available style sheets with `plt.style.available` . Once we have selected a style sheet we can apply it with the function `plt.style.use()` . I have a soft-spot for the `seaborn` style, and so would apply it as:

```
plt.style.use('seaborn-v0_8')

plt.plot(temp, ice_creams)
plt.xlabel("Temperature (degrees)")
plt.ylabel("Ice creams consumed")
```

After a call to `plt.style.use()` , *all* the subsequent plots we produce in the current Python session will have the specified style applied (Figure 4.5).

4.2 Other plot types

Line and scatter plots aren't the only type of plots which we can produce with Matplotlib. In fact, Matplotlib supports a great number of different plot types.

Function	Description
<code>plt.hist(x)</code>	Plot a histogram from count or density data <code>x</code>
<code>plt.bar(x, h)</code>	Make a barplot with bars of height <code>h</code> at locations <code>x</code>
<code>plt.boxplot(x)</code>	Produce a boxplot (LQ, median, UQ, outliers) of <code>x</code>
<code>plt.pie(x)</code>	Create a pie chart from count data or percentages <code>x</code>

4.3 Plotting with `DataFrame`s

The Pandas `DataFrame` has a `.plot()` method which provides a convenient way to produce Matplotlib-based plots. The `.plot()` method allows a number of different plot types, which we specify with the `kind` argument. The argument passed to `kind` can be any of: `'line'` , `'bar'` , `'barh'` , `'hist'` , `'box'` , `'kde'` , `'density'` , `'area'` , `'pie'` , `'scatter'` or `'hexbin'` .

```
# Import course package
import jrpyintroduction as jrpy

# Load movies data
movies = jrpy.data.load('movies')
```

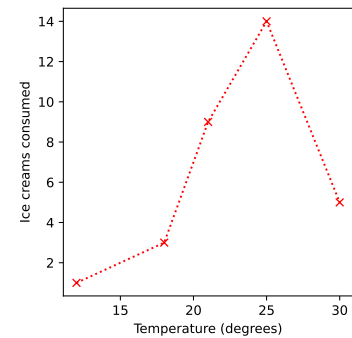


Figure 4.4: Passing the third optional format argument to `plt.plot()` as `'x:r'` .

Character	Description
<code>+</code>	plus marker
<code>o</code>	circle marker
<code>x</code>	cross marker
<code>*</code>	star marker
<code>.</code>	point marker

Table 4.1: Marker styles available to `plt.plot()` .

Character	Description
<code>-</code>	solid line style
<code>--</code>	dashed line style
<code>-.</code>	dash-dot line style
<code>:</code>	dotted line style

Table 4.2: Line styles and their shorthand representation.

Character	Colour
<code>b</code>	blue
<code>g</code>	green
<code>r</code>	red
<code>c</code>	cyan
<code>m</code>	magenta
<code>y</code>	yellow
<code>k</code>	black

Table 4.3: Basic colours and shorthand names as understood by `plt.plot()` .

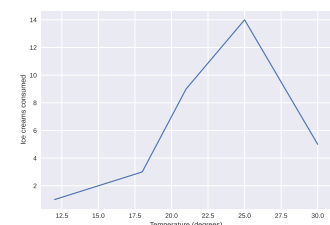


Figure 4.5: The `plt.style.use()` function allows to conveniently alter multiple aspects of our plot's aesthetics.

```
# Take a random sample of 2000 rows
movies = movies.sample(2000)

# Plot the year column against the rating column
movies.plot(x='year', y='rating', kind='scatter')
```

We can also use the `.plot()` method to colour data-points by some third variable: again specified as some column of our `DataFrame`, and passed to the `c` argument. By default the `.plot()` method will choose a rather unsatisfactory grayscale colourmap to represent this third variable. We can specify that a different colourmap is used with the `cmap` argument (the `alpha` argument can also be used to add transparency):

```
movies.plot(x='year', y='rating', kind='scatter',
           c='budget', cmap='viridis', alpha=0.4)
```

Colourmaps

Matplotlib offers a number of different colourmaps for you to choose from (see: matplotlib.org/tutorials/colors/colormaps). However, not all colourmaps were created equal, and some colourmaps have better visual-perceptive qualities than others. The long and short of it is this avoid dated colourmaps such as the (unfortunately) ever-popular `jet` colourmap, and instead look to use more modern, better designed colourmaps such as the `viridis` colourmap.

4.4 Saving plots

Cool, so we can create our plots in Python. But how do we get our plots out of Python and into our groundbreaking research report? Well, that's easy with the `plt.savefig()` function. We tell `plt.savefig()` where to save our plot and under what name by specifying a path:

```
plt.plot(x, y)
plt.title('Science')
plt.xlabel('x')
plt.ylabel('y')

# Save plot as a pdf file with name science_plot
plt.savefig('/home/linux/Documents/jr/science.pdf')
```

the `plt.savefig()` function will infer the file type you'd like to save your plot as from the file extension of the passed path.

File extensions—an aside

As a general rule:

- Don't save plots which contain text (axis labels, titles, legends,...) as `.jpg` files; `.jpg`'s compression algorithm is known to perform poorly on text. the `.jpg` format should be reserved for photographs.
- Save plots to a vector format (eg. `.pdf`) where possible. Vector graphics

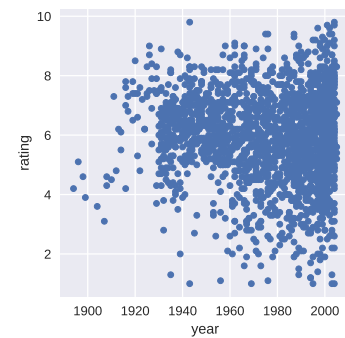


Figure 4.6: A figure produced by using the `.plot()` method of the Pandas

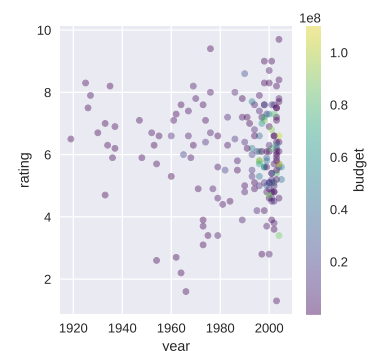
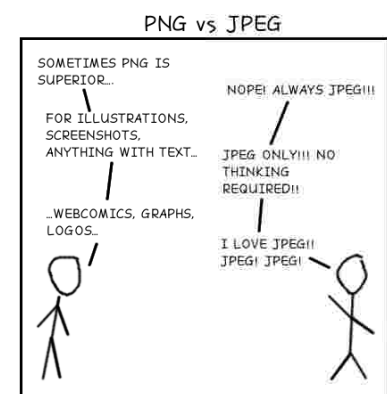


Figure 4.7: Using the `c` argument to colour points by a third value. The `cmap` argument is used to change the colourmap used, and the `alpha` argument adds transparency.

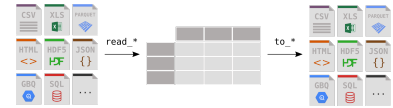


allow a viewer to zoom-in on a plot as far as they'd like, without *ever* encountering any pixelation.

- When a vector format is inappropriate (interoperability with other software, file size concerns,...) use the .png format. .png's compression algorithm performs better on text than .jpg's.

Reading and Writing Data

We have now seen how we can summarise, manipulate and visualise data in Python. However, so far, all this data has come from *within* Python itself, whereas in practice our data tends to come from *external* sources. The Pandas package supports reading data from, and writing data to, a great number of different file types.



5.1 CSV files

Comma separated value (CSV) files are a common format used to store data. In such files data values are arranged into rows and columns, where individual values are separated by commas (or by some other delimiter, such as tabs, or spaces).

Reading

The function `pd.read_csv()` provides a convenient way to read such data into Python:

```
import pandas as pd

# Read in the data in the file 'data.csv'
df = pd.read_csv('data.csv')
df
#>   x  y  z
#> 0  1  2  1
#> 1  2  3  4
#> 2  3  5  9
#> 3  4  7 16
#> 4  5 11 25
```

x,	y,	z
1,	2,	1
2,	3,	4
3,	5,	9
4,	7,	16
5,	11,	25

Table 5.1: Contents of the `data.csv` file.

By default the `pd.read_csv()` function assumes that the values to be read-in are separated by commas (`sep=','`). If the values are separated by some other delimiter we may specify this with the `sep` argument. If, for example, the values are separated by semi-colons: `pd.read_csv('data.csv', sep=';')`, or if the values were separated by tabs: `pd.read_csv('data.csv', sep='\t')`.

Writing

As well as *reading* CSV files into Pandas `DataFrame`s, we can also *write* Pandas `DataFrame`s out to CSV files. We can do so with the `.to_csv()` method. As with the `pd.read_csv()` function, the `.to_csv()` takes an *optional* `sep` argument (again defaulting to the comma):

```
# Write data to a file 'pandas_data.csv', and
# separate the values with semi-colons
df.to_csv('pandas_data.csv', sep=';')
```

```
; x; y; z
0; 1; 2; 1
1; 2; 3; 4
2; 3; 5; 9
3; 4; 7; 16
4; 5; 11; 25
```

Table 5.2: Contents of the `pandas_data.csv` file.

5.2 Excel files

Pandas also provides functionality for reading and writing from the ever-present and increasingly unavoidable Excel Spreadsheet.

Reading

Reading an Excel formatted file into Python is, as a Windows user, as simple as:

```
pd.read_excel("exhel_sheet.xlsx")
```

replacing `"exhel_sheet.xlsx"` with the name of your Excel file.

However, non-Windows users must burden themselves with another step if they desire to read (2010) Excel files. Namely, they must install the `openpyxl` package and read their Excel files as:

```
data = pd.read_excel("bill_sheets.xlsx",
                    engine="openpyxl")
```

As with `pd.read_csv()`, `pd.read_excel()` has a large number of optional arguments which may be used to control how the Excel data should be parsed.

Writing

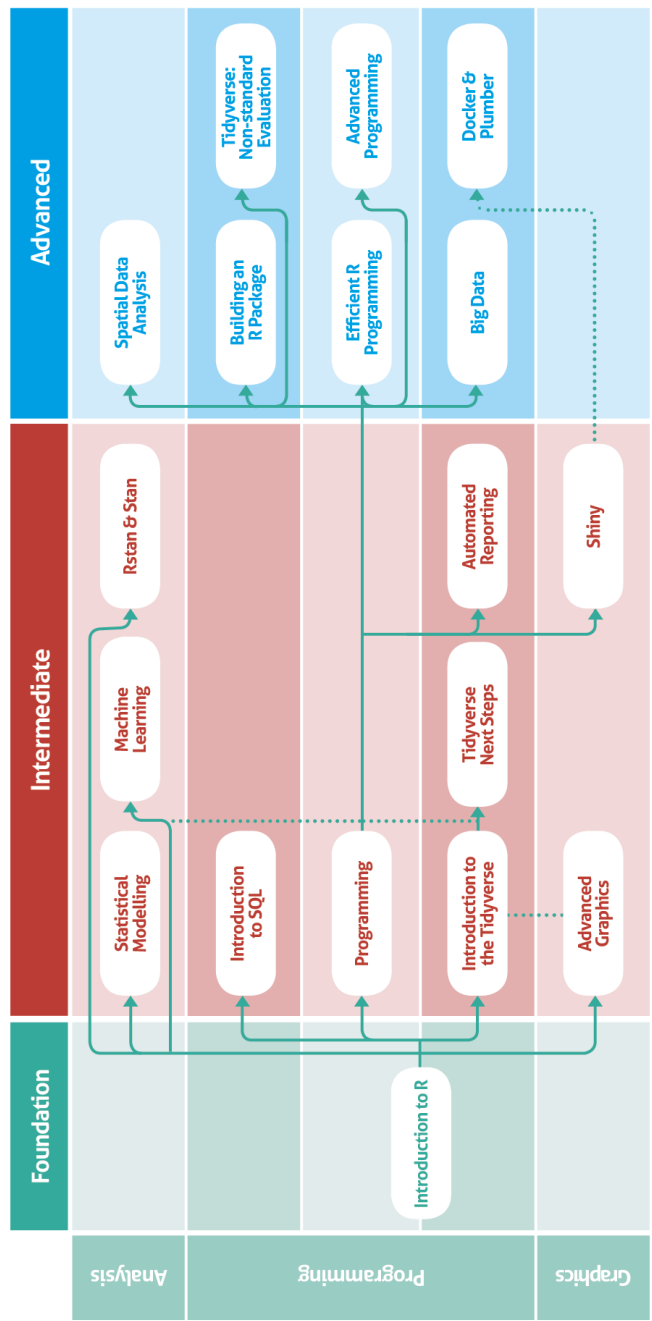
As with `pd.read_csv()`, `pd.read_excel()` has a corresponding write method `df.to_excel()`.

5.3 Other formats

Pandas also supports the reading and writing of other file formats.

Data format	Read	Write
json	<code>pd.read_json()</code>	<code>df.to_json()</code>
hdf	<code>pd.read_hdf()</code>	<code>df.to_hdf()</code>
sas	<code>pd.read_sas()</code>	—
spss	<code>pd.read_spss()</code>	—
stata	<code>pd.read_stata()</code>	<code>df.to_stata()</code>

Training Course Dependencies





OUR PARTNERS